

SOFTWARE EVOLUTION



PREPARED BY:

Muhammad Asim

Ph# - +923066010010

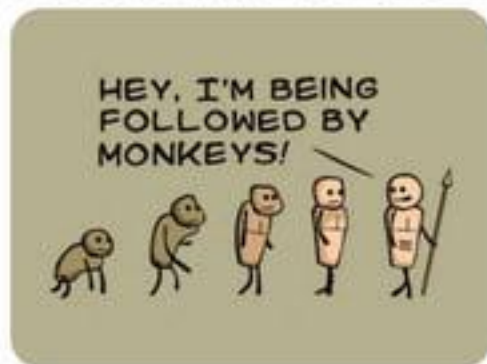


WHAT IS EVOLUTION ?

- the gradual development of something.

*Evolution is what happens
while you're busy
making other plans."*

- The term evolution does not refer to changes that occur in an individual within its life time .Instead it refers to the changes in the characteristics of population over the course of generation.
- E.g. the Human Evolution Theory.



IMPORTANCE OF EVOLUTION

- ❖ Organizations have huge investments in their software systems - they are critical business assets.
- ❖ To maintain the value of these assets to the business, they must be changed and updated.
- ❖ Companies prefer evolution on new developments.



SOFTWARE EVOLUTION

- ***Evolution*** is what actually happens to the software
- There are two things in software evolution.
 - i. Software change.
 - ii. Software maintenance.



SOFTWARE CHANGE

- Software change is unavoidable
 - New requirements emerge when the software is used
 - The business environment changes
 - Errors must be repaired
 - New equipment must be accommodated
 - The performance or reliability may have to be improved.



CHANGE PREDICTION

- Predicting the number of changes requires and understanding of the relationships between a system and its environment.
- Tightly coupled systems require changes whenever the environment is changed.
- Factors influencing this relationship are
 - Number and complexity of system interfaces.
 - The business processes where the system is used.



STRATEGIES FOR CHANGING SOFTWARE SYSTEMS

- Software maintenance
 - Architectural evolution
 - Software re-engineering
- **Software maintenance**
 - Changes are made in response to changed requirements but the fundamental software structure is stable
 - **Architectural transformation**
 - The architecture of the system is modified.
 - **Software re-engineering**
 - No new functionality is added to the system but it is restructured and reorganised to facilitate future changes



SOFTWARE MAINTENANCE

- Maintenance does not normally involve major changes to the system's architecture
- The system requirements are likely to change while the system is being developed because the environment is changing. Therefore a delivered system won't meet its requirements!
- Systems are tightly coupled with their environment. When a system is installed in an environment it changes that environment and therefore changes the system requirements.
- Systems **MUST** be maintained therefore if they are to remain useful in an environment.

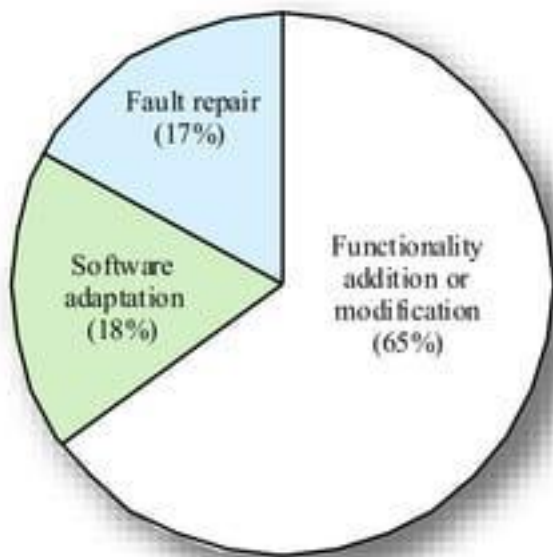


TYPES OF MAINTENANCE

- *Corrective*
 - Maintenance to repair software faults
- *Adaptive*
 - Maintenance to adapt software to a different operating environment
- *Perfective*
 - Maintenance to add to or modify the system's functionality



DISTRIBUTION OF MAINTENANCE EFFORT

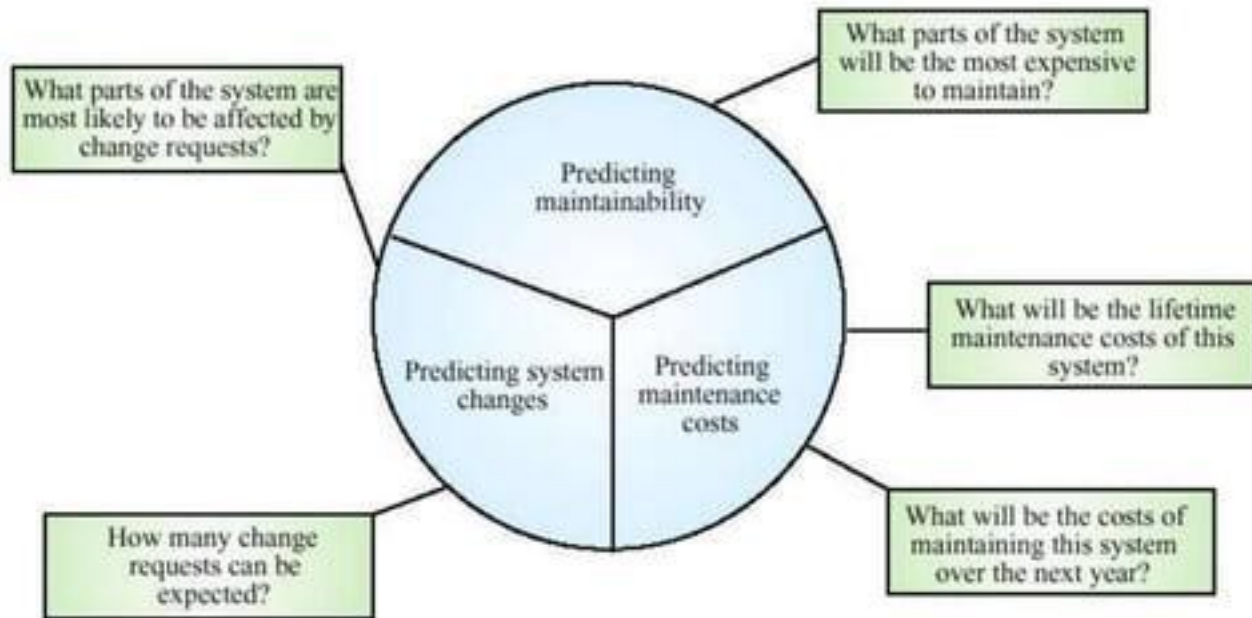


MAINTENANCE PREDICTION

- Maintenance prediction is concerned with assessing which parts of the system may cause problems and have high maintenance costs
 - Change acceptance depends on the maintainability of the components affected by the change;
 - Implementing changes reduces its maintainability;
 - Maintenance costs depend on the number of changes and costs of change depend on maintainability.



MAINTENANCE PREDICTION

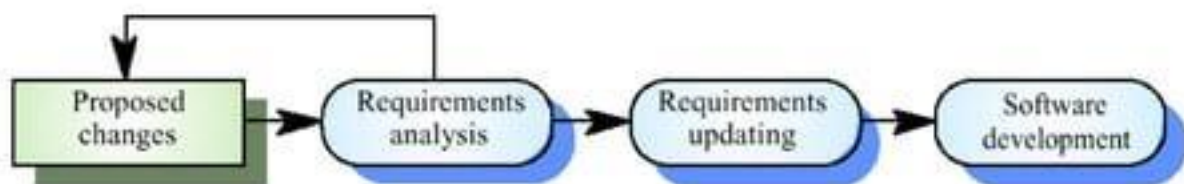


CHANGE REQUESTS

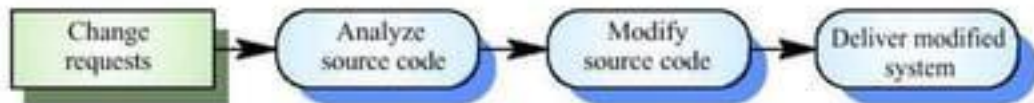
- Change requests are requests for system changes from users, customers or management
- In principle, all change requests should be carefully analysed as part of the maintenance process and then implemented
- In practice, some change requests must be implemented urgently
 - Fault repair
 - Changes to the system's environment
 - Urgently required business changes



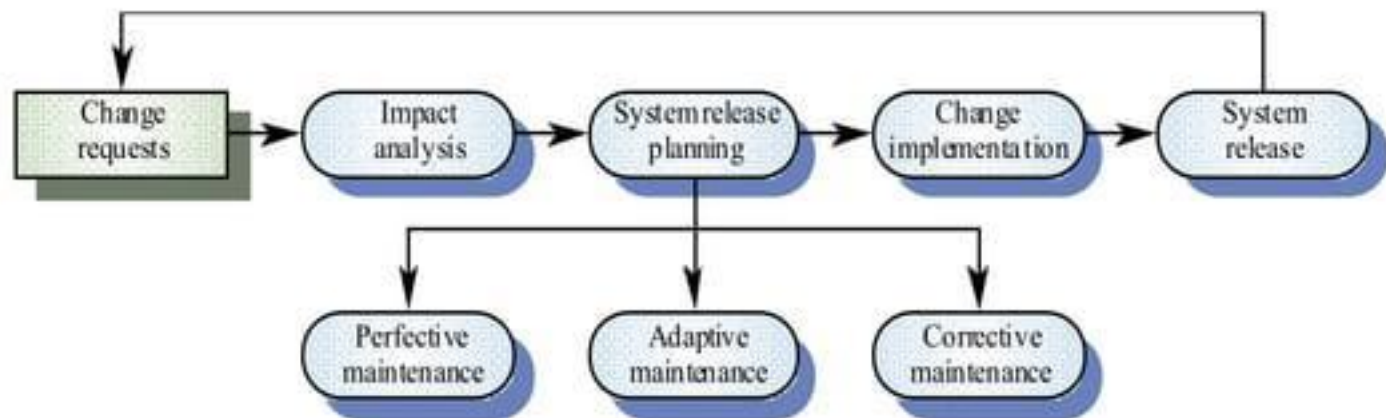
CHANGE IMPLEMENTATION



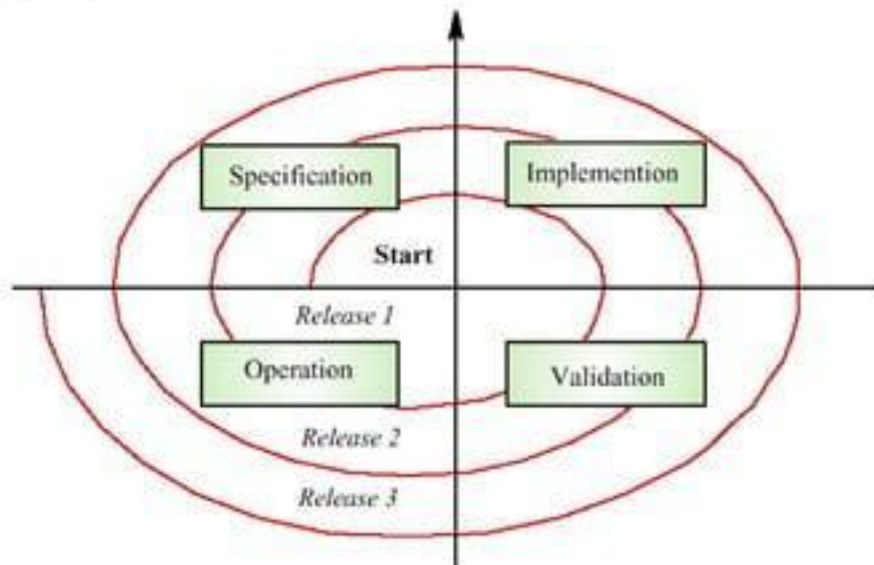
EMERGENCY REPAIR



SOFTWARE MAINTENANCE PROCESS



SPIRAL MAINTENANCE MODEL



MAINTENANCE COSTS

- Usually greater than development costs
- Increases as software is maintained. Maintenance corrupts the software structure so makes further maintenance more difficult.
- Ageing software can have high support costs (e.g. old languages, compilers etc.)



MAINTENANCE COST FACTORS

- Team stability
 - Maintenance costs are reduced if the same staff are involved with them for some time
- Contractual responsibility
 - The developers of a system may have no contractual responsibility for maintenance so there is no incentive to design for future change
- Staff skills
 - Maintenance staff are often inexperienced and have limited domain knowledge
- Program age and structure
 - As programs age, their structure is degraded and they become harder to understand and change



EVOLUTIONARY SOFTWARE

- Rather than think of separate development and maintenance phases, evolutionary software is software that is designed so that it can continuously evolve throughout its lifetime



PROGRAM EVOLUTION DYNAMICS

- **Program evolution dynamics** is the study of the processes of system change.
- After major empirical studies, Lehman and Belady proposed that there were a number of 'laws' which applied to all systems as they evolved.
- There are sensible observations rather than laws. They are applicable to large systems developed by large organisations. Perhaps less applicable in other cases.



LEHMAN'S LAWS

Law	Description
Continuing change	A program that is used in a <u>real-world environment</u> must necessarily change, or else become <u>progressively less useful in that environment</u> .
Increasing complexity	As an evolving program changes, its structure <u>tends to become more complex</u> . Extra resources must be <u>devoted to preserving and simplifying the structure</u> .
Large program evolution	System attributes such as <u>size</u> , <u>time between releases</u> , and the number of reported errors is approximately <u>invariant</u> for each system release.
Organizational stability	Over a program's lifetime, its <u>rate of development is approximately constant and independent of the resources devoted to system development</u> .



LEHMAN'S LAWS (CONT'D)

Law	Description
Conservation of familiarity	Over the lifetime of a system, <u>the incremental change in each release is approximately constant.</u>
Continuing growth	<u>The functionality offered by systems has to continually increase to maintain user satisfaction.</u>
Declining quality	<u>The quality of systems will decline unless they are modified to reflect changes in their operational environment.</u>
Feedback system	Evolution processes <u>incorporate multiagent, multiloop feedback systems and you have to treat them as feedback systems to achieve significant product improvement.</u>



ARCHITECTURAL EVOLUTION

- There is a need to convert many legacy systems from a centralised architecture to a client-server architecture
- Change drivers
 - Hardware costs. Servers are cheaper than mainframes
 - User interface expectations. Users expect graphical user interfaces
 - Distributed access to systems. Users wish to access the system from different, geographically separated, computers



DISTRIBUTION FACTORS

Factor	Description
Business importance	Returns on the investment of distributing a legacy system depend on its importance to the business and how long it will remain important. If distribution provides more efficient support for stable business processes then it is more likely to be a cost-effective evolution strategy.
System age	The older the system the more difficult it will be to modify its architecture because previous changes will have degraded the structure of the system.
System structure	The more modular the system, the easier it will be to change the architecture. If the application logic, the data management and the user interface of the system are closely intertwined, it will be difficult to separate functions for migration.
Hardware procurement policies	Application distribution may be necessary if there is company policy to replace expensive mainframe computers with cheaper servers. .



DISTRIBUTION OPTIONS

- The more that is distributed from the server to the client, the higher the costs of architectural evolution
- The simplest distribution model is UI distribution where only the user interface is implemented on the server
- The most complex option is where the server simply provides data management and application services are implemented on the client



DISTRIBUTION OPTION SPECTRUM

Server: Interaction control
Data validation
Services
Database

Server: Services
Database

Server: Database



Client: Presentation

Client: Presentation
Interaction control
Data validation

Client: Presentation
Interaction control
Data validation
Services



Increasing cost
and effort

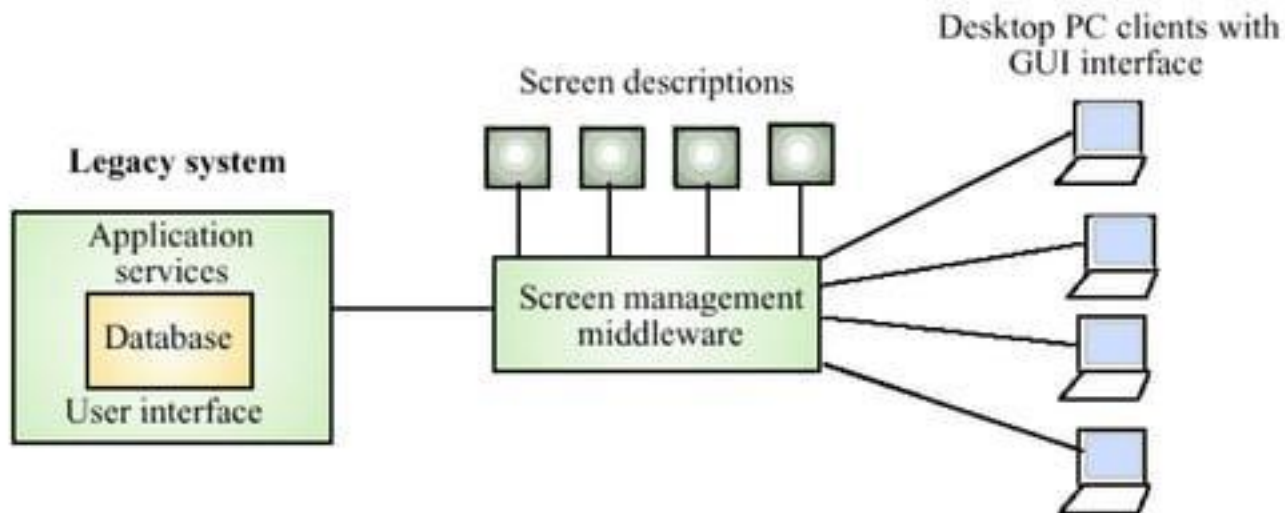


USER INTERFACE DISTRIBUTION

- UI distribution takes advantage of the local processing power on PCs to implement a graphical user interface
- Where there is a clear separation between the UI and the application then the legacy system can be modified to distribute the UI
- Otherwise, screen management middleware can translate text interfaces to graphical interfaces



USER INTERFACE DISTRIBUTION



USER INTERFACE MIGRATION STRATEGIES

Strategy	Advantages	Disadvantages
Implementation using the window management system	Access to all UI functions so no real restrictions on UI design Better UI performance	Platform dependent May be more difficult to achieve interface consistency
Implementation using a web browser	Platform independent Lower training costs due to user familiarity with the WWW Easier to achieve interface consistency	Potentially poorer UI performance Interface design is constrained by the facilities provided by web browsers



LEGACY SYSTEM EVOLUTION

- Organizations that rely on legacy systems must choose a strategy for evolving these systems.
 - Scrap the system completely and modify business processes so that it is no longer required;
 - Continue maintaining the system;
 - Transform the system by re-engineering to improve its maintainability;
 - Replace the system with a new system.
- The strategy chosen should depend on the system quality and its business value.



LEGACY SYSTEM CATEGORIES

- Low quality, low business value

These systems should be scrapped.

- Low-quality, high-business value

These make an important business contribution but are expensive to maintain. Should be re-engineered or replaced if a suitable system is available.

- High-quality, low-business value

Replace with COTS, scrap completely or maintain.

- High-quality, high business value

Continue in operation using normal system maintenance.

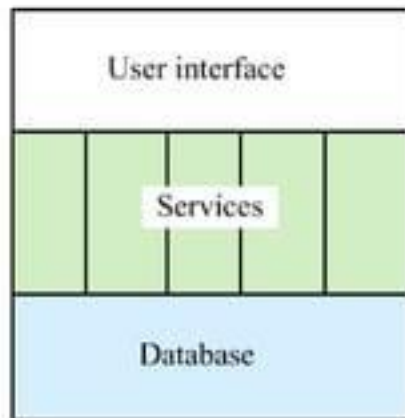


LEGACY SYSTEM STRUCTURE

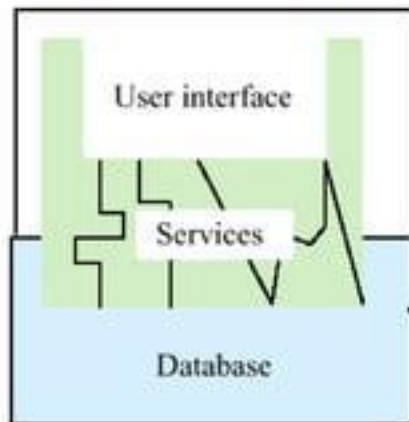
- Ideally, for distribution, there should be a clear separation between the user interface, the system services and the system data management
- In practice, these are usually intermingled in older legacy systems



LEGACY SYSTEM STRUCTURES



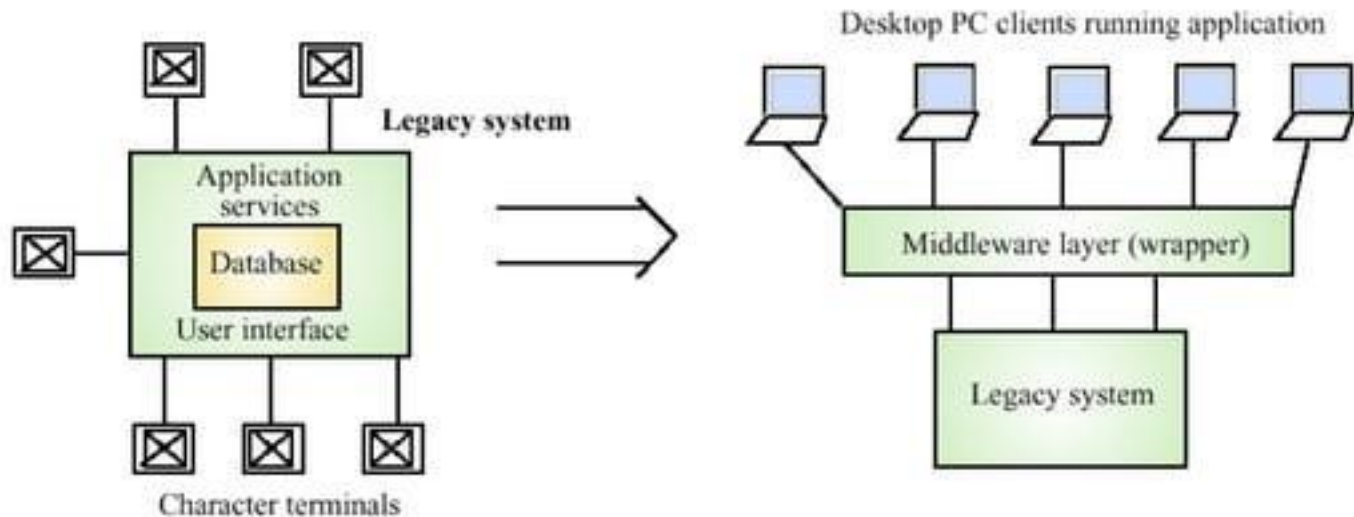
Ideal model for distribution



Real legacy systems



LEGACY SYSTEM DISTRIBUTION



SYSTEM RE-ENGINEERING

- Re-structuring or re-writing part or all of a legacy system without changing its functionality.
- Applicable where some but not all sub-systems of a larger system require frequent maintenance.
- Re-engineering involves adding effort to make them easier to maintain. The system may be re-structured and re-documented.



ADVANTAGES OF RE-ENGINEERING

- Reduced risk
- There is a high risk in new software development. There may be development problems, staffing problems and specification problems.
- Reduced cost
- The cost of re-engineering is often significantly less than the costs of developing new software.



FORWARD AND RE-ENGINEERING



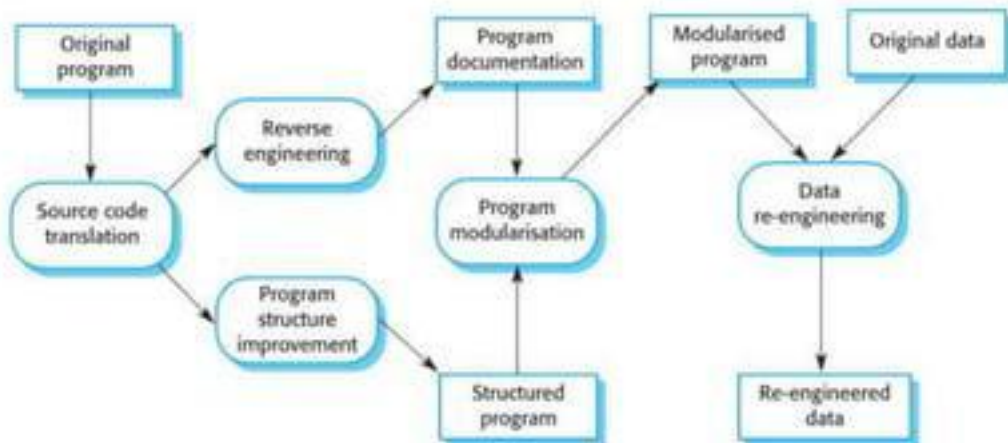
Forward engineering



Software re-engineering



THE RE-ENGINEERING PROCESS



REENGINEERING PROCESS ACTIVITIES

- Source code translation

Convert code to a new language.

- Reverse engineering

Analyze the program to understand it;

- Program structure improvement

Restructure automatically for understandability;

- Program modularization

Reorganize the program structure;

- Data reengineering

Clean-up and restructure system data.

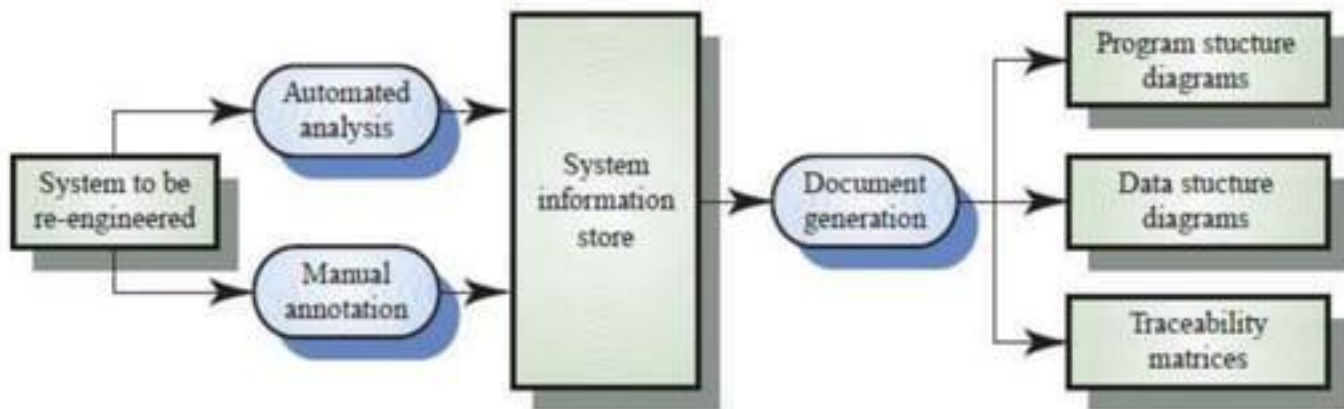


REVERSE ENGINEERING

- Analyzing software with a view to understanding its design and specification
- May be part of a re-engineering process but may also be used to re-specify a system for re-implementation
- Builds a program data base and generates information from this.
- Program understanding tools (browsers, cross-reference generators, etc.) may be used in this process



THE REVERSE ENGINEERING PROCESS



REVERSE ENGINEERING

- Reverse engineering often precedes re-engineering but is sometimes worthwhile in its own right.
- The design and specification of a system may be reverse engineered so that they can be an input to the requirements specification process for the system's replacement.
- The design and specification may be reverse engineered to support program maintenance



ANY QUESTIONS



**Thank
You!!!**

for lending your ears ^^

